

API versioning

1 What API Versioning Really Means

API versioning is a strategy to:

Allow your API to evolve without breaking existing clients.

Think long-term:

- Mobile apps update slowly.
- Third-party integrations may never update.
- Enterprise clients may depend on your API for years.

If you change behavior without versioning → production disaster.

What Is a Breaking Change (Deep Understanding)

A breaking change is:

Any change that forces existing consumers to modify their code.

Example 1 – Field Rename (Breaking)

v1:

```
{  
  "name": "Prajwal"  
}
```

v2:

```
{  
  "fullName": "Prajwal Jadhav"  
}
```

Frontend expecting name → crashes.

Example 2 – Data Type Change (Breaking)

v1:

```
{  
  "age": "25"  
}
```

v2:

```
{  
  "age": 25  
}
```

If frontend parses as string → breaks.

Example 3 – Removing Field (Breaking)

Removing anything is dangerous.

Example 4 – Adding Required Field (Breaking)

If you add a required request body field:

v1:

```
{  
  "email": "abc@gmail.com"  
}
```

v2:

```
{  
  "email": "abc@gmail.com",  
  "phone": "9999999999" // now required  
}
```

Old clients won't send phone → request fails.

3 API Versioning Strategies (In Detail)

There are 4 real-world strategies.

1. URI Versioning (Most Popular)

[/api/v1/users](#)

[/api/v2/users](#)

Why It's Popular

- Very explicit
- Easy for frontend teams
- Easy to test
- Easy to debug
- Easy to cache

Express Example

```
app.use('/api/v1', require('./routes/v1'));  
app.use('/api/v2', require('./routes/v2'));
```

Folder structure:

```
routes/  
  v1/  
    user.routes.js  
  v2/  
    user.routes.js
```

Each version can have separate controllers.

2. Header Versioning (Cleaner URLs)

URL stays same:

GET /api/users

Client sends:

API-Version: 2

Or:

Accept: application/vnd.myapp.v2+json

Express Example

```
app.use((req, res, next) => {  
  req.version = req.headers['api-version'] || '1';
```

```
next();
});

app.get('/api/users', (req, res) => {
  if (req.version === '1') {
    return res.json({ name: "Prajwal" });
  }
  if (req.version === '2') {
    return res.json({ fullName: "Prajwal Jadhav" });
  }
});
```

Pros

- Cleaner URLs
- Looks more RESTful
- Keeps endpoint stable

Cons

- Harder to test manually
- Harder for beginners
- Debugging slightly harder
- Caching complexity

👉 Used in large enterprise systems.



3. Query Parameter Versioning

[/api/users?version=1](#)

Why It's Weak

- Easy to ignore
- Caching issues
- Feels like hack

- Not expressive

Only use if legacy system forces you.



4. Content Negotiation (Advanced)

Client sends:

[Accept: application/vnd.company.user-v2+json](#)

Server decides representation format.

Very powerful, but complex.

Mostly used in:

- Enterprise systems
- Complex RESTful architectures



Major vs Minor Versioning

Important concept.

You don't bump version for everything.

Major Version (v1 → v2)

- Breaking changes
- Structural change
- Removing fields

Minor Change

- Add optional fields
- Improve performance
- Add new endpoint

Often you don't change version for minor updates.

- Keep services shared
- Only change transformation layer

Example:

```
// service.js
function getUser() {
  return {
    id: 1,
    firstName: "Prajwal",
    lastName: "Jadhav"
  };
}

// v1 controller
res.json({ name: user.firstName });

// v2 controller
res.json({ fullName: user.firstName + " " + user.lastName });
```

Only response shape differs.

Business logic stays same.

This prevents maintenance hell.

Versioning Mistakes (That Kill Startups)

1. Not Versioning From Day 1
2. Copy-Paste Entire Codebase
3. Supporting Too Many Versions Forever

API Endpoints

What Is an API Endpoint?

An API endpoint is:

A specific URL + HTTP method that performs a specific operation.

It represents a contract between client and server.

Example:

GET /api/v1/users

POST /api/v1/users

GET /api/v1/users/123

DELETE /api/v1/users/123

Each combination of:

- **URL**
- **HTTP Method**
- **Version**
- **Headers**
- **Request body structure**

= One endpoint.

What Makes a Good Endpoint?

A well-designed endpoint:

- **Is predictable**
- **Is resource-oriented**
- **Uses proper HTTP verbs**
- **Is versioned**
- **Has consistent naming**

Internal Anatomy of an Endpoint

When request hits:

Client → Route → Middleware → Controller → Service → Database

Flow:

1. **Router matches endpoint**

2. Middleware checks auth
3. Controller validates request
4. Service handles business logic
5. Database responds
6. Response formatted
7. Sent back

API GATEWAY

What Is an API Gateway?

An API Gateway is:

A single entry point for multiple backend services.

Instead of client calling 10 microservices directly:

Client → Gateway → Microservices

What Does API Gateway Actually Do?

It's not just routing.

It can handle:

1 Routing

/api/users → user-service

/api/orders → order-service

2 Authentication

Instead of every service verifying JWT:

Gateway verifies once.

3 Rate Limiting

Prevent abuse:

- 100 requests per minute per user
 - Protect system
-

4 Logging & Monitoring

Centralized request tracking.

5 Load Balancing

Distribute traffic across service instances.

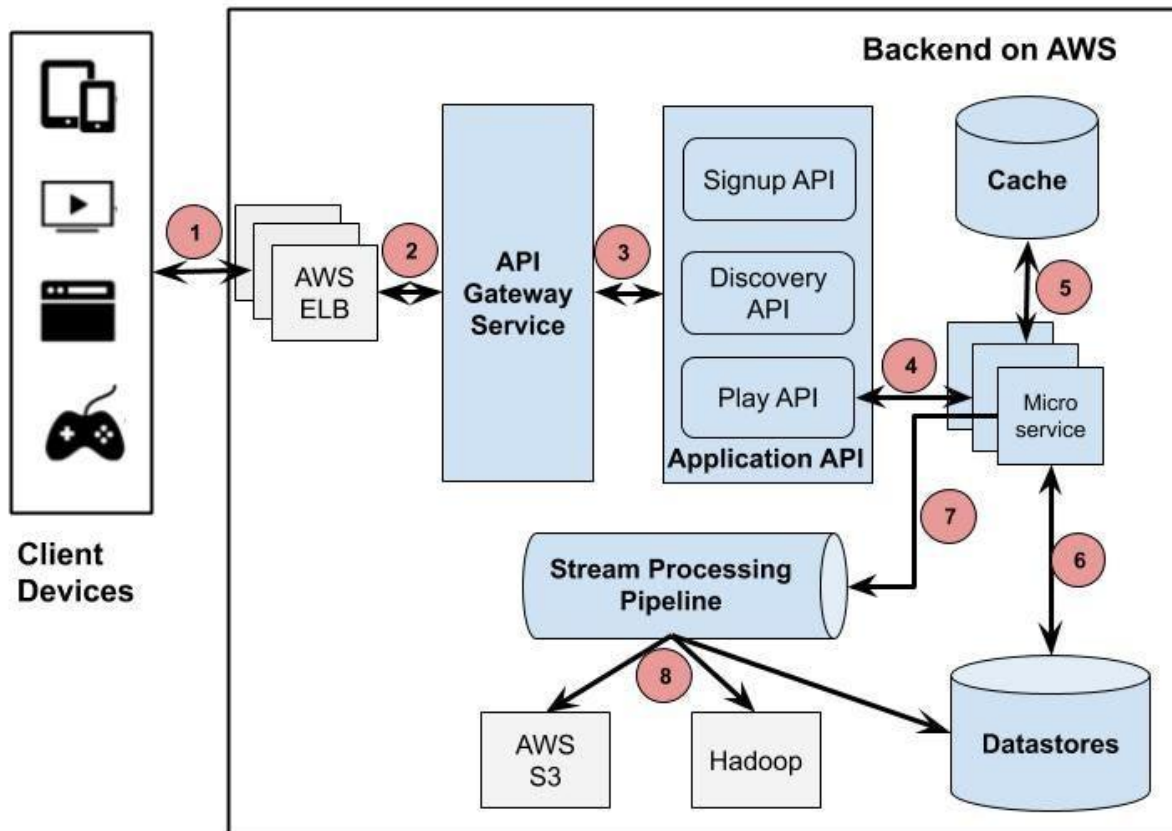
6 Response Aggregation

Client needs:

- User profile
- Order history
- Recommendations

Instead of 3 requests:

Gateway fetches from 3 services and merges response.



Types of API Gateways

1 Edge Gateway

Exposed to internet.

Handles:

- Auth
- Rate limiting
- SSL
- Routing

2 Internal Gateway

Used between microservices.

Handles:

- Service discovery

- Traffic control

Endpoints vs Gateway (Clear Difference)

Feature	Endpoint	API Gateway
What it is	Specific route	Entry layer
Scope	Single service	Multiple services
Handles business logic?	Yes	No (usually)
Handles routing?	Yes	Yes (global)
Handles auth?	Maybe	Usually yes

Difference between endpoint and route?

Answer:

- A route is the backend configuration that maps a URL to a handler.
- An endpoint is the exposed API contract that the client interacts with.

In simple terms:

Route = server-side mapping

Endpoint = public interface

How do you secure API endpoints?

Answer:

Multiple layers:

- Authentication (JWT, OAuth)
- Authorization (RBAC)
- Input validation
- Rate limiting

- HTTPS
- Logging & monitoring

Security should not rely only on frontend validation.

When should you create a new API version?

Answer:

Only when there is a breaking change.

What is API deprecation?

Answer:

Deprecation is the process of phasing out an old API version.

What problems does API Gateway solve?

Answer:

It solves:

- Client complexity
- Duplicate authentication logic
- Security exposure
- Rate limiting control
- Centralized logging
- Request aggregation

It simplifies microservice architecture.

Difference between API Gateway and Load Balancer?

This is VERY commonly asked.

Answer:

API Gateway

Routes based on URL paths

Handles auth, rate limiting

Can transform requests/responses

Application-level

Load Balancer

Distributes traffic evenly

Just distributes traffic

Does not modify requests

Network-level

Load balancer balances traffic.

Gateway manages API behavior.

What should NOT be inside an API Gateway?

Answer:

Gateway should not contain:

- Business logic
- Database calls
- Heavy processing

It should remain:

- Thin
- Fast
- Stateless

Otherwise it becomes a bottleneck.

How does API Gateway work with JWT authentication?

Answer:

Flow:

1. Client sends JWT in Authorization header.
2. Gateway verifies token.
3. If valid → forwards request to service.
4. Services trust gateway.

This avoids every microservice verifying tokens separately.

When do you NOT need an API Gateway?

Answer:

You don't need it when:

- You have a simple monolithic backend
- Small application
- Single service architecture

Gateway is useful when microservices grow.